A Technical Introduction to Retrieval augmented generation

Section 1

Applications Retrieval-augmented generation is used in applications where generated responses need to be grounded in external or frequently updated information. Commonly cited use cases include search engines, question-answering systems, customer support chatbots, enterprise knowledge assistants, content generation, recommendation systems, retail and e-commerce, and industrial or manufacturing workflows. In healthcare, RAG has been studied as a way to ground large language model outputs in external medical knowledge sources, although reviews have noted continuing challenges around evaluation, ethics, and clinical reliability.

Section 2

Pre-training the retriever using the Inverse Cloze Task (ICT), a technique that helps the model learn retrieval patterns by predicting masked text within documents. Supervised retriever optimization aligns retrieval probabilities with the generator model's likelihood distribution. This involves retrieving the top-k vectors for a given prompt, scoring the generated response's perplexity, and minimizing KL divergence between the retriever's selections and the model's likelihoods to refine retrieval. Reranking techniques can refine retriever performance by prioritizing the most relevant retrieved documents during training.

Section 3

Retrieval-augmented generation (RAG) is a technique that enables large language models (LLMs) to retrieve and incorporate new information from external data sources. With RAG, LLMs first refer to a specified set of documents, then respond to user queries. These documents supplement information from the LLM's pre-existing training data. This allows LLMs to use domain-specific and/or updated information that is not available in the training data. For example, this helps LLM-based chatbots access internal company data or generate responses based on authoritative sources. RAG improves LLMs by incorporating information retrieval before generating responses. Unlike LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources. According to Ars Technica, "RAG is a way of improving LLM performance, in essence by blending the LLM process with a web search or other document look-up process to help LLMs stick to the facts." This method helps reduce AI hallucinations, which have caused chatbots to describe policies that don't exist, or recommend nonexistent legal cases to lawyers that are looking for citations to support their arguments. RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs. Beyond efficiency gains, RAG also allows LLMs to include sources in their responses, so users can verify the cited sources. This provides greater transparency, as users can cross-check retrieved content to ensure accuracy and relevance. The term retrieval-augmented generation (RAG) was introduced in a 2020 paper that described combining a parametric language model with a non-parametric external memory accessed through retrieval at inference time.

Section 4

Process Retrieval-augmented generation (RAG) enhances large language models (LLMs) by incorporating an information-retrieval mechanism that allows models to access and utilize additional data beyond their original training set. Ars Technica notes that "when new information becomes available, rather than having to retrain the model, all that's needed is to augment the model's external knowledge base with the updated information" ("augmentation"). IBM states that "in the generative phase, the LLM draws from the augmented prompt and its internal representation of its training data to synthesize" an answer.

Section 5

By redesigning the language model with the retriever in mind, a 25-time smaller network can get comparable perplexity as its much larger counterparts. Because it is trained from scratch, this method (Retro) incurs the high cost of training runs that the original RAG scheme avoided. The hypothesis is that by giving domain knowledge during training, Retro needs less focus on the domain and can devote its smaller weight resources only to language semantics. The redesigned language model is shown here. It has been reported that Retro is not reproducible, so modifications were made to make it so. The more reproducible version is called Retro++ and includes in-context RAG.

Section 6

Encoder These methods focus on the encoding of text as either dense or sparse vectors. Sparse vectors, which encode the identity of a word, are typically dictionary-length and contain mostly zeros. Dense vectors, which encode meaning, are more compact and contain fewer zeros. Various enhancements can improve the way similarities are calculated in the vector stores (databases).

Section 7

Typically, the data to be referenced is converted into LLM embeddings, numerical representations in the form of a large vector space. RAG can be used on unstructured (usually text), semi-structured, or structured data (for example knowledge graphs). These embeddings are then stored in a vector database to allow for document retrieval.

Given a user query, a document retriever is first called to select the most relevant documents that will be used to augment the query. This comparison can be done using a variety of methods, which depend in part on the type of indexing used. The model feeds this relevant retrieved information into the LLM via prompt engineering of the user's original query. Newer implementations (as of 2023) can also incorporate specific augmentation modules with abilities such as expanding queries into multiple domains and using memory and self-improvement to learn from previous retrievals. Finally, the LLM can generate output based on both the query and the retrieved documents. Some models incorporate extra steps to improve output, such as the re-ranking of retrieved information, context selection, and fine-tuning.

Section 8

Fixed length with overlap. This is fast and easy. Overlapping consecutive chunks helps to maintain semantic context across chunks. Syntax-based chunks can break the document up into sentences. Libraries such as spaCy or NLTK can also help. File format-based chunking. Certain file types have natural chunks built in, and it's best to respect them. For example, code files are best chunked and vectorized as whole functions or classes. HTML files should leave <table> or base64 encoded <img> elements intact. Similar considerations should be taken for pdf files. Libraries such as Unstructured or LangChain can assist with this method.

Section 9

Performance improves by optimizing how vector similarities are calculated. Dot products enhance similarity scoring, while approximate nearest neighbor (ANN) searches improve retrieval efficiency over K-nearest neighbors (KNN) searches. Accuracy may be improved with Late Interactions, which allow the system to compare words more precisely after retrieval. This helps refine document ranking and improve search relevance. Hybrid vector approaches may be used to combine dense vector representations with sparse one-hot vectors, taking advantage of the computational efficiency of sparse dot products over dense vector operations. Other retrieval techniques focus on improving accuracy by refining how documents are selected. Some retrieval methods combine sparse representations, such as SPLADE, with query expansion strategies to improve search accuracy and recall.